

# Projekt - Saper

Autorzy:

Emilia Wojtiuk

Opis:

Saper jest prostą grą komputerową stworzoną w 1981 roku, w którą prawie każdy właściciel komputera grał. Sama w sobie jest niezwykle prosta w obsłudze jak i w graniu, ponieważ głównym zadaniem jest odkrywanie zakrytych pól.

Założenia gry:

- Gracz ustawia wielkość planszy oraz ilość min
- Gracz odkrywa kolejne zakryte pola do zakończenia rozgrywki
- Na każdym z odkrytych pól napisana jest liczba min, które bezpośrednio stykają się z danym odkrytym polem (1~8).
- Przy trafieniu na liczbę zostanie odkryte tylko wybrane pole.
- W przypadku pola wokół którego nie znajduje się żadna miny (0), zostaną odkryte wszystkie sąsiednie puste pola (0), włącznie z najbliższym graniczącym polem z liczbą (1~8).
- Istnieje możliwość zabezpieczenia pola przed odkryciem przy użyciu flagi
- Przy przerwaniu rozgrywki jest możliwość zapisania aktualnego stanu gry i późniejszego jego wczytania

(Rozgrywka jest zapisywana w Play\_Board.json według odgórnie wybranych grafik w liście\_obrazków)

Zakończenie rozgrywki:

- Odkrycie miny skutkuje natychmiastową przegraną
- Aby wygrać należy odkryć wszystkie pola niebędące miną

## Część pierwsza projektu ~ Podsumowanie Gry w terminalu:

Instrukcja rozpoczęcia rozgrywki:

Po przejściu do sam dół programu znajdującego się w saper.py zobaczymy main().

Początkowo wybieramy liczbę wierszy i kolumn jak i bomb(gdzie nieodpowiednie wpisanie wierszy i kolumn wypisze się nam błąd natomiast w przypadku bomb zostanie wybrana randomowa liczba min).

```
def main():  
    lista_obrazkow = [" ", "1", "2", "3", "4", "5", "6", "7", "8", "B", "F", "X"]  
    new_game = int(input("New_game?:\nYes - 1, No - Any other button\n"))  
    wiersze = 4  
    kolumny = 3  
    bomby = 5
```

Następnie uruchamiamy nasz program w saper.py. Uzupełniamy pliki z których mamy czytać dane o planszy, przy błędnym wpisaniu zostają użyte odgórnie przypisane pliki w URL'u. Jeżeli pragniemy zagrać na nowej planszy wybieramy 1, w przeciwnym wypadku zostanie uruchomiona poprzednia plansza z poprzednio wybranymi wartościami(wierszy, kolumn i bomb).

```
New_game?:
Yes - 1, No - Any other button
1
[0, 1, 1]
[1, 2, 11]
[11, 4, 2]
[11, 11, 1]
['X', 'X', 'X']
['X', 'X', 'X']
['X', 'X', 'X']
['X', 'X', 'X']
myszka:
l - lewy przycisk, p - prawy przycisk
```

Należy odpowiednio wybierać lewy(„l”) albo prawy(„p”) przycisk w terminalu a następnie wiersz wraz z kolumną do odkrycia/oflagowania/odflagowania.

```
l          <-wpisana wartość
wiersz: 2  <-wpisana wartość
kolumna: 0 <-wpisana wartość
```

Nasza rozgrywka będzie polegać na zasadach znanych z oryginalnego Sapera – gdzie wciśnięcie na bombę skutkuje pokazaniem wszystkich bomb i zakończeniem rozgrywki. Jeżeli chcemy zakończyć rozgrywkę sami, wciskamy ctrl+c, co zakończy naszą grę w terminalu, którą zawsze możemy wczytać gdy przy pytaniu New\_game? wybierzemy jakikolwiek inny numer niż 1.

```
['X', 'X', 'X']
['X', 'X', 'B']
['B', 'X', 'X']
['B', 'B', 'X']
You loose
```

Kolejne działania będą skutkować odpowiednio:

| Kliknięcie na cyfrę: | Kliknięcie na „zero”: | Oflagowanie:    | You win:        |
|----------------------|-----------------------|-----------------|-----------------|
| l                    | l                     | p               | l               |
| wiersz: 2            | wiersz: 0             | wiersz: 3       | wiersz: 3       |
| kolumna: 1           | kolumna: 0            | kolumna: 0      | kolumna: 3      |
| ['X', 'X', 'X']      | ['0', '1', 'X']       | ['0', '1', 'X'] | ['0', '1', '1'] |
| ['X', 'X', 'X']      | ['1', '2', 'X']       | ['1', '2', 'X'] | ['1', '2', 'X'] |
| ['X', '4', 'X']      | ['X', '4', 'X']       | ['X', '4', 'X'] | ['X', '4', '2'] |
| ['X', 'X', 'X']      | ['X', 'X', 'X']       | ['F', 'X', 'X'] | ['F', 'X', '1'] |
|                      |                       |                 | You win         |

Wykorzystywane pliki:

- saper.py
- test\_saper.py (testy)
- play\_board.json
- board.json

Saper.py uproszczone działanie funkcji:

Class Dict:

- Przy wywołaniu tej funkcji automatycznie zostaje stworzona mapa z randomowym rozkładem min, przy informacjach takich jak: liczba wierszy, kolumn, bomb i czy istnieje ewentualnie ogólnie stworzona mapa samych bomb;
- `make_d()` - randomowo rozkłada miny, których ilość poprzednio wybraliśmy
- `neighbours()` - zostaje wywołana w czasie stworzenia naszej planszy, gdzie jej celem jest znalezienie ilości sąsiadujących bomb

Class Play\_Board:

- `lista_ob` = lista obrazków do graficznego przedstawienia
- `outcome` - czy kontynuujemy rozgrywkę
- `usedF` - użyte flagi
- `win` - wygraliśmy? (True/False)
- potrzebne informacje - liczba wierszy, kolumn, bomb(sprawdzanie wygranej), `lista_o(brazków)`(dla wglądu graficznego w mainie), `New_game`(czy tworzyć nową planszę)
- `print_raw_map_raw` - wypisanie kolejnych wierszy planszy z informacjami o sąsiadach, bombach i pustych miejscach
- `print_raw_map` - również wypisywanie, jednak tutaj wypisywana w terminalu jest nasza aktualna rozgrywka
- `make_play_board` - tworzy czystą planszę z zakrytymi wszystkimi polami, zapisuje w `play_board.json`
- `click` - w zależności od typu kliku przenosi nas do funkcji `right_click` albo `left_click` - sprawdza czy nie wygraliśmy (`you_win()`)
- `lef_click` - w zależności informacji na obu planszach odkrywa dane pola
- `right_click` - w zależności informacji na obu planszach ustawia odpowiednio flagi
- `you_win()` - poprzez iterację jeżeli na jakimś innym miejscu niż bomba znajduje się flaga albo zakryte miejsce - kontynuujemy rozgrywkę
- `show_bombs` - odkrywa bomby i kończy rozgrywkę

<https://gitlab-stud.elka.pw.edu.pl/ewojtiuk/pipr-projekt.git>